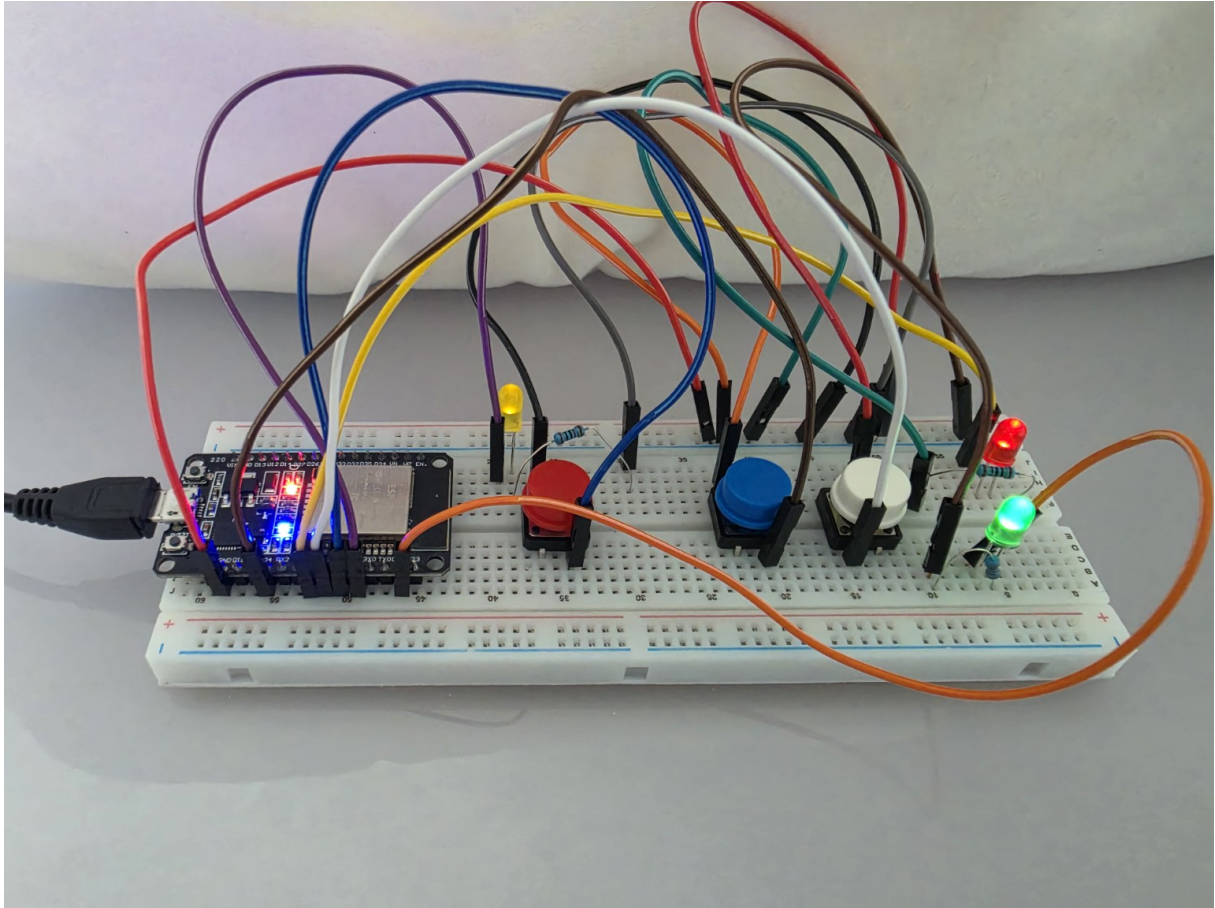


ESP32 Smart Home Beginner Project

MicroPython • ESP32 • Buttons • LEDs • Smart Home Basics

By Ahmad Azroun

This document starts with the German version first, followed by the complete English version below.



Deutsche Dokumentation

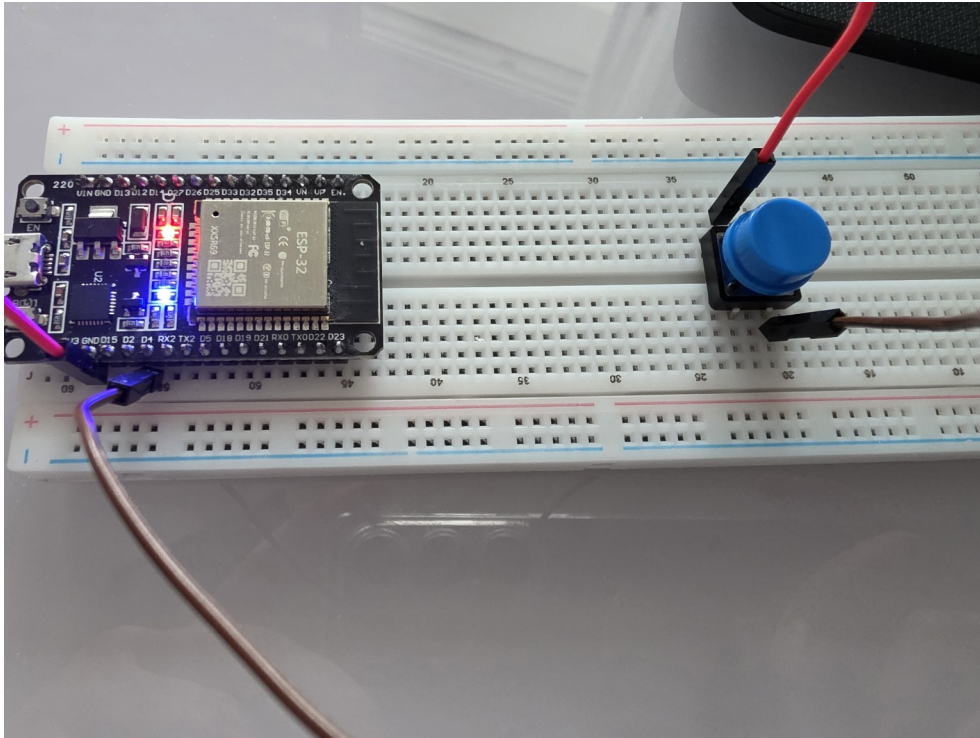
Dieses Projekt dokumentiert die ersten praktischen Schritte im Bereich IoT und Smart Home mit ESP32 und MicroPython. Das Projekt begann mit der Steuerung der internen LED und entwickelte sich später zu mehreren LEDs, Tastern und Widerständen.

Verwendete Komponenten

Komponente	Beschreibung
ESP32 DevKit V1	Microcontroller-Board
Breadboard	Schaltungsaufbau
Taster	Benutzereingaben
LEDs	Visuelle Ausgänge
Widerstände	Strombegrenzung und Schutz
Jumper Kabel	Elektrische Verbindungen
MicroPython	Programmiersprache

1. Interne LED mit einem Taster

Im ersten Schritt wurde ein einzelner Taster verwendet, um die interne LED des ESP32 zu steuern. Der Taster ist an D4 / GPIO4 angeschlossen, während die interne LED über GPIO2 gesteuert wird.



```
from machine import Pin
import time

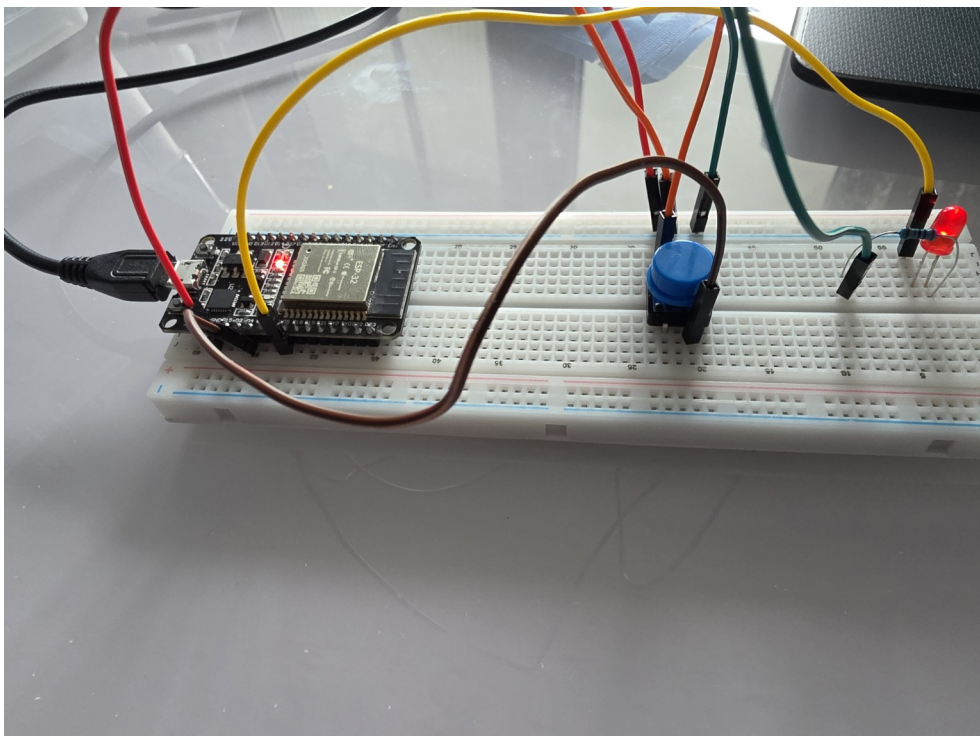
button = Pin(4, Pin.IN, Pin.PULL_UP)
led_builtin = Pin(2, Pin.OUT)

while True:
    if button.value() == 0:
        led_builtin.on()
    else:
        led_builtin.off()

    time.sleep(0.05)
```

2. Externe LED mit einem Taster

Danach wurde eine externe LED mit einem Widerstand verwendet. Der Widerstand schützt die LED und begrenzt den Stromfluss.



```
from machine import Pin
```

```
import time

button = Pin(4, Pin.IN, Pin.PULL_UP)
led = Pin(5, Pin.OUT)

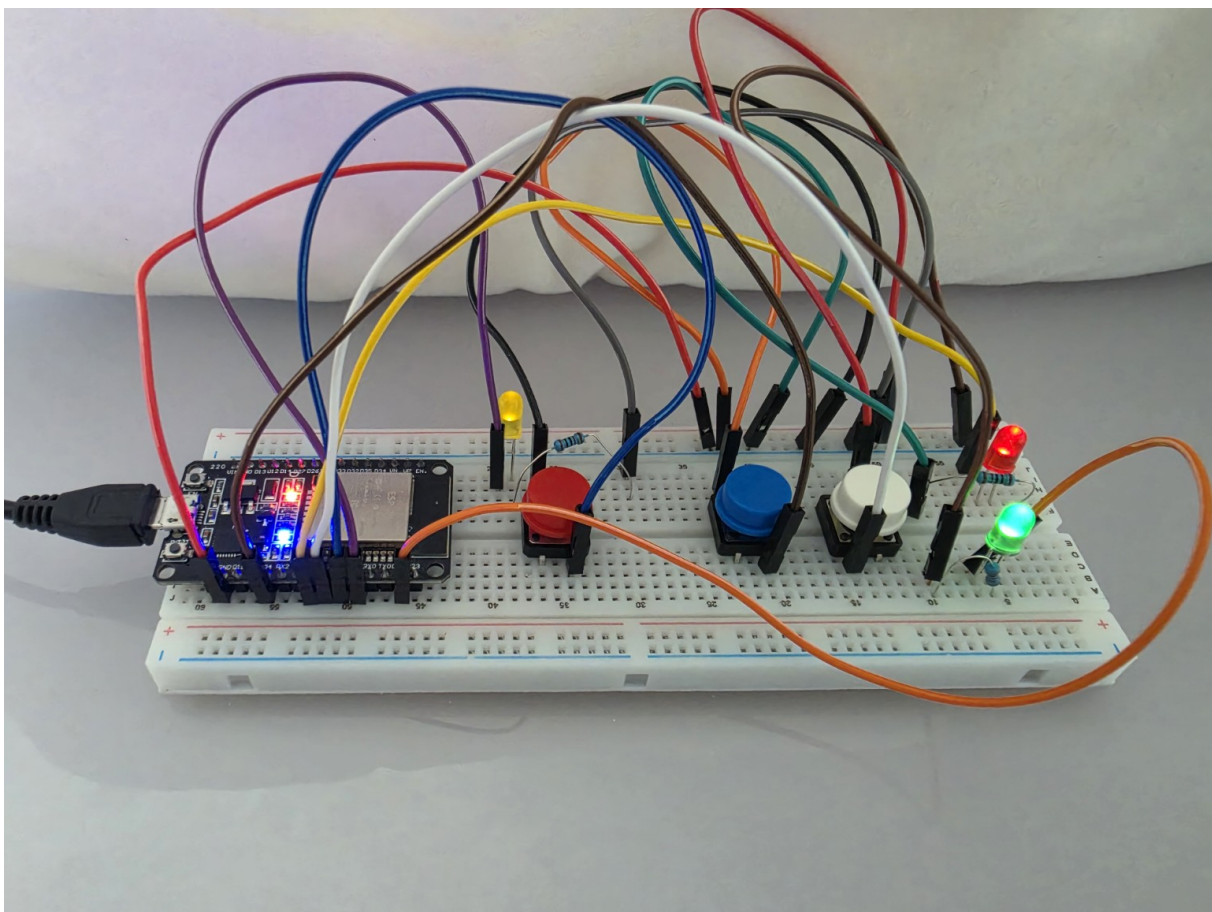
while True:
    if button.value() == 0:
        led.on()
    else:
        led.off()

    time.sleep(0.05)
```

3. Vollständiges Projekt

Die finale Version verwendet: Drei Taster Interne und externe LEDs Einen Taster zur Steuerung von zwei LEDs gleichzeitig Standalone-Ausführung mit main.py Widerstände zum Schutz der LEDs

Taster	Funktion
Button 1 (GPIO4)	Steuert zwei externe LEDs
Button 2 (GPIO19)	Steuert eine externe LED
Button 3 (GPIO18)	Steuert die interne LED



```
from machine import Pin
import time

button1 = Pin(4, Pin.IN, Pin.PULL_UP)

led1 = Pin(5, Pin.OUT)
led_extra = Pin(22, Pin.OUT)

button2 = Pin(19, Pin.IN, Pin.PULL_UP)
led2 = Pin(21, Pin.OUT)

button3 = Pin(18, Pin.IN, Pin.PULL_UP)
led_builtin = Pin(2, Pin.OUT)

state1 = 0
state2 = 0
state3 = 0

last1 = 1
last2 = 1
last3 = 1

while True:

    c1 = button1.value()

    if last1 == 1 and c1 == 0:
        state1 = 1 - state1
```

```
        led1.value(state1)
        led_extra.value(state1)

last1 = c1

c2 = button2.value()

if last2 == 1 and c2 == 0:
    state2 = 1 - state2
    led2.value(state2)

last2 = c2

c3 = button3.value()

if last3 == 1 and c3 == 0:
    state3 = 1 - state3
    led_builtin.value(state3)

last3 = c3

time.sleep(0.05)
```

English Documentation

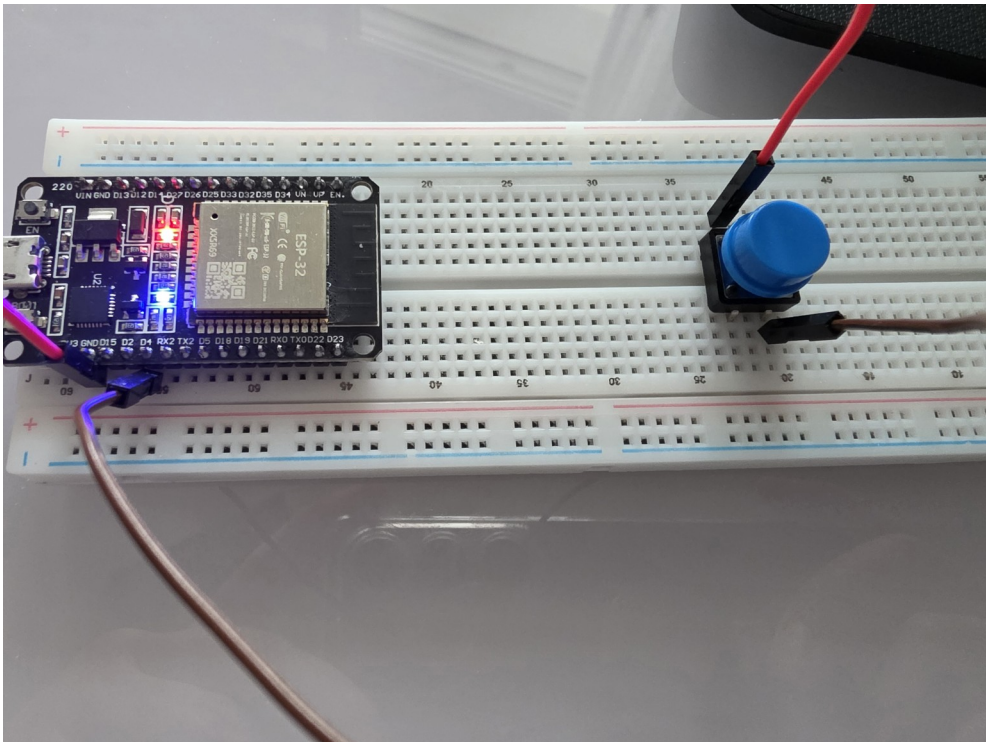
This project documents the first practical steps in IoT and Smart Home development using ESP32 and MicroPython. The project started with controlling the internal LED and later evolved into controlling multiple LEDs using buttons and resistors.

Components Used

Component	Description
ESP32 DevKit V1	Microcontroller board
Breadboard	Circuit prototyping
Push Buttons	User input
LEDs	Visual output indicators
Resistors	Current limiting and protection
Jumper Wires	Electrical connections
MicroPython	Programming language

1. Internal LED with One Button

The first stage of the project used one push button to control the internal LED of the ESP32. The button was connected to D4 / GPIO4, while the internal LED was controlled using GPIO2.



```
from machine import Pin
import time

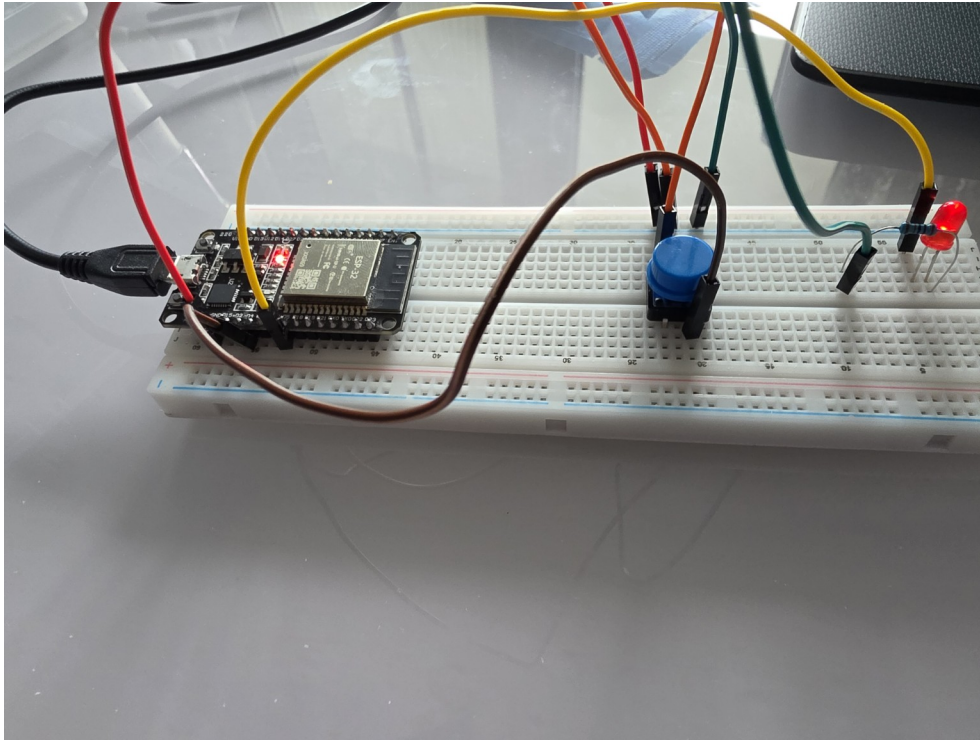
button = Pin(4, Pin.IN, Pin.PULL_UP)
led_builtin = Pin(2, Pin.OUT)

while True:
    if button.value() == 0:
        led_builtin.on()
    else:
        led_builtin.off()

    time.sleep(0.05)
```

2. External LED with One Button

The next step added an external LED with a resistor. The resistor protects the LED and limits the current flow.



```
from machine import Pin
import time

button = Pin(4, Pin.IN, Pin.PULL_UP)
led = Pin(5, Pin.OUT)

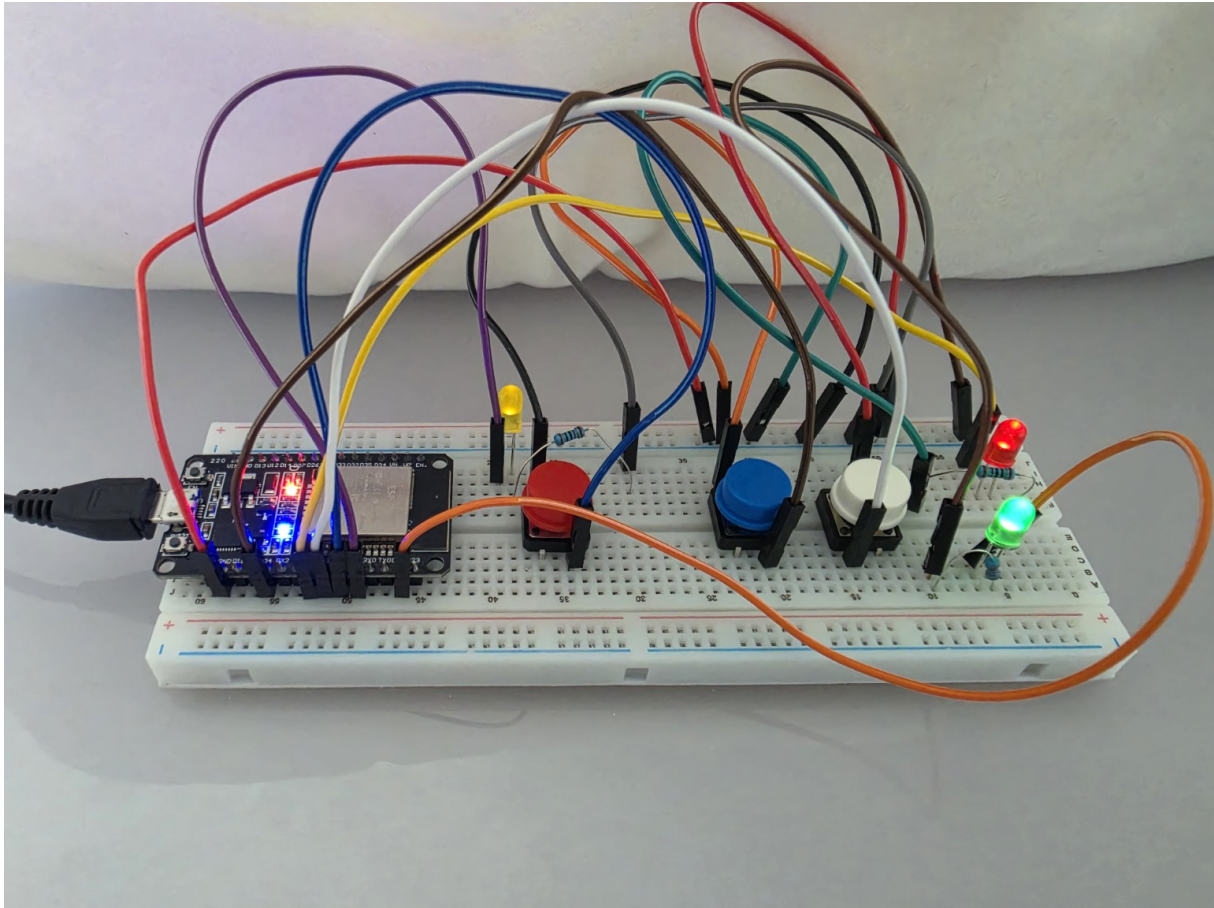
while True:
    if button.value() == 0:
        led.on()
    else:
        led.off()

    time.sleep(0.05)
```

3. Complete Final Project

The final version of the project includes: Three push buttons Internal and external LEDs One button controlling two LEDs together Standalone execution using main.py Resistors for LED protection

Button	Function
Button 1 (GPIO4)	Controls two external LEDs
Button 2 (GPIO19)	Controls one external LED
Button 3 (GPIO18)	Controls the internal LED



```
from machine import Pin
import time

button1 = Pin(4, Pin.IN, Pin.PULL_UP)

led1 = Pin(5, Pin.OUT)
led_extra = Pin(22, Pin.OUT)

button2 = Pin(19, Pin.IN, Pin.PULL_UP)
led2 = Pin(21, Pin.OUT)

button3 = Pin(18, Pin.IN, Pin.PULL_UP)
led_builtin = Pin(2, Pin.OUT)

state1 = 0
state2 = 0
state3 = 0

last1 = 1
last2 = 1
last3 = 1

while True:

    c1 = button1.value()

    if last1 == 1 and c1 == 0:
        state1 = 1 - state1
        led1.value(state1)
        led_extra.value(state1)

    last1 = c1

    c2 = button2.value()

    if last2 == 1 and c2 == 0:
        state2 = 1 - state2
        led2.value(state2)

    last2 = c2

    c3 = button3.value()

    if last3 == 1 and c3 == 0:
        state3 = 1 - state3
        led_builtin.value(state3)

    last3 = c3

    time.sleep(0.05)
```

Future Improvements

WiFi control Bluetooth control Fan and relay control Temperature and humidity sensors Smart energy systems Solar energy integration AI-based Smart Home automation

Ahmad Azroun